

hwk2/paush2.vhd

```

1  -----
2  --
3  --  SH-2 Program Memory Access Unit
4  --
5  --  This is an implementation of a Program Memory Access Unit for the SH-2 it
6  --  uses the generic memory unit. The generic address unit takes in a number of
7  --  sources and offsets and allows for pre and post increment and decrement.
8  --  The PAU uses this to generate the appropriate address for instruction
9  --  fetches and updates the PC. PC + 4 for branching will need to be changed
10 --  when pipelined.
11 --
12 --  Entities included are:
13 --    PAUSH2 - the program access unit for SH2
14 --
15 --  Revision History:
16 --    9 Apr 26  Simone Shevchuk      Initial revision.
17 --    11 Apr 26 Simone Shevchuk      Update PC and PR added
18 --    13 Apr 26 Simone Shevchuk      VBR moved to DAU, shifter entity removed, explicit mapping for arrays
19 --    30 Apr 26 Simone Shevchuk      Add PR as an output which will go to reg array for STS
20 --    1 May 26  Simone Shevchuk      Change PC to in increment by 2
21 --    8 May 26  Simone Shevchuk      PC+4 branch base and delay slot support
22 --
23  -----
24
25 --
26 --  PAUSH2
27 --
28 --  This is a program memory access unit for the SH-2. It uses the generic
29 --  memory access unit to generate the appropriate address for instruction
30 --  fetches. It outputs along the address bus the address for the next instruction,
31 --  this can also be fed directly into the PR to store the return address.
32 --  Pre/post is set to pre by MAU standards meaning use the original address and
33 --  no inc/dec since the inc/dec is never used since offset applies changes.
34 --  The PAU has an input for the offset to be added to the PC for jump.
35 --  A shifter is used to shift the offset left by 1 bit to multiply by 2 which
36 --  some instructions require. All offsets are sign extended in the control unit
37 --  before coming in. The possible offsets are the sign extended shifted value
38 --  from instructions (0) (note shifting is done in the PAU but sign extension
39 --  done in the control unit); value from a register (1), zero (2), and one (3)
40 --  which is actually equivalent to 4 bytes for an increment. The value for an
41 --  Address source can be current PC (0), PR(1), datadatabus(2) or PC+=4(3) for
42 --  normal increments, returns, handling interrupts and absolute jumps, and
43 --  relative branches respectively. The PC holds the value of the current
44 --  instruction being decoded and is updated to the next instruction address on
45 --  every clock cycle. Everything with + 4 will change with pipelining.
46 --
47 --
48 --  Inputs:
49 --    src_sel      - source to use (log srccnt bits)
50 --    datadata_src - value from VBR and offset for reset vector for interrupt handling
51 --    addr_off     - offset given from instruction
52 --    addr_reg     - offset given from register
53 --    offset_sel   - offset to use (log offsetcnt bits)
54 --    pr_sel       - signal to write to PR with current PC or register
55 --    delay_slot   - asserted by control unit during delayed branch decode
56 --    clock        - the system clock
57 --    reset        - the system reset signal
58 --
59 --  Outputs:
60 --    address_out  - address bus (32 bits), next instruction address to fetch
61 --    PR          - output PR value to be written to register file for JSR and RTS instructions
62 --
63
64 library ieee;
65 use ieee.std_logic_1164.all;

```

```

66 use work.array_type_pkg.all;
67
68 entity PAUSH2 is
69
70     port(
71         src_sel      : in  integer range 3 downto 0;
72         datadata_src : in  std_logic_vector(31 downto 0);
73         addr_off     : in  std_logic_vector(31 downto 0);
74         addr_reg     : in  std_logic_vector(31 downto 0);
75         offset_sel   : in  integer range 3 downto 0;
76         pr_sel       : in  integer range 3 downto 0;
77         delay_slot   : in  std_logic;
78         clock         : in  std_logic;
79         reset        : in  std_logic;
80         address_out   : buffer std_logic_vector(31 downto 0);
81         PR            : out  std_logic_vector(31 downto 0) -- output PR for STS
82     );
83
84 end PAUSH2;
85
86 architecture behavioral of PAUSH2 is
87
88     constant ZERO    : std_logic_vector(31 downto 0) := (others => '0'); -- constant for 0 value
89     constant ONE     : std_logic_vector(31 downto 0) := x"00000002";
90     -- constant for 1 value, which is 2 bytes since instructions are word aligned and incrementing
91     -- by 1 means incrementing by 2 bytes to get to the next instruction
92     constant FOUR    : std_logic_vector(31 downto 0) := x"00000004";
93     -- constant for 4 value, used for branch targets since they are PC + 4 + offset
94
95     signal PC        : std_logic_vector(31 downto 0) := ZERO;
96     signal PC_plus4   : std_logic_vector(31 downto 0);
97     signal pc4_carry  : std_logic_vector(32 downto 0);
98     signal PC_plus2   : std_logic_vector(31 downto 0);
99     signal pc2_carry  : std_logic_vector(32 downto 0);
100    signal next_PC    : std_logic_vector(31 downto 0) := ZERO;
101    signal Shift2Res   : std_logic_vector(31 downto 0);
102    -- result of shifting offset left by 1 bit to multiply by 2 for jump instructions with shifted offsets
103    signal mau_addr    : std_logic_vector(31 downto 0);
104    signal branch_target : std_logic_vector(31 downto 0) := ZERO;
105    signal delay_active : std_logic := '0';
106    signal src_array    : std_logic_array(3 downto 0)(31 downto 0); -- array of sources for MAU
107
108 begin
109
110     Regs : entity work.MemUnit
111     generic map (
112         srcCnt    => 4,
113         offsetCnt => 4,
114         wordsize  => 32
115     )
116     port map (
117         AddrSrc    => src_array,
118         SrcSel     => src_sel,
119         AddrOff    => (0 => Shift2Res, 1 => addr_reg, 2 => ZERO, 3 => ONE),
120         OffsetSel  => offset_sel,
121         IncDecSel  => '0',
122         IncDecBit  => 0,
123         PrePostSel => '0',
124         Address    => mau_addr,
125         AddrSrcOut => open
126     );
127
128     Shift2Res <= addr_off(30 downto 0) & '0';
129
130     -- PC + 4 for branch target base (target = PC(pointing to second instrr) + offset)
131     pc4_carry(0) <= '0';
132     PC4Adder: for i in PC_plus4'range generate

```

```

133     begin
134         PC4Bit: entity work.MAUAdderBit port map (
135             A => PC(i), B => FOUR(i), Ci => pc4_carry(i),
136             S => PC_plus4(i), Co => pc4_carry(i + 1)
137         );
138     end generate;
139
140     -- PC + 2 for fetching the delay slot instruction
141     pc2_carry(0) <= '0';
142     PC2Adder: for i in PC_plus2'range generate
143     begin
144         PC2Bit: entity work.MAUAdderBit port map (
145             A => PC(i), B => ONE(i), Ci => pc2_carry(i),
146             S => PC_plus2(i), Co => pc2_carry(i + 1)
147         );
148     end generate;
149
150     -- if in the delay slot MAU clculating branch target still need to grab next
151     -- instruction at PC+2, target then saved in register
152     -- if in a delay-active portion then fetch from saved branch target
153     -- otherwise regular MAU output (sequential or hold)
154     address_out <= PC_plus2      when delay_slot = '1' else
155                     branch_target when delay_active = '1' else
156                     mau_addr;
157
158     next_PC <= address_out;
159
160     -- index 3 is PC+4 used as branch base (branch target = PC(pointing to second instrr) + offset)
161     src_array <= (0 => PC, 1 => PR, 2 => datadata_src, 3 => PC_plus4);
162
163     process(clock)
164     begin
165         if rising_edge(clock) then
166             if reset = '0' then
167                 PC          <= ZERO;
168                 PR          <= ZERO;
169                 branch_target <= ZERO;
170                 delay_active <= '0';
171             else
172                 if pr_sel = 1 then
173                     PR <= PC_plus4; -- BSR/JSR save PC+4 as return address
174                 elsif pr_sel = 2 then
175                     PR <= addr_reg;
176                 elsif pr_sel = 3 then
177                     PR <= datadata_src;
178                 end if;
179
180                 if delay_slot = '1' then
181                     branch_target <= mau_addr;
182                     delay_active <= '1';
183                 else
184                     delay_active <= '0';
185                 end if;
186
187                 PC <= next_PC;
188             end if;
189         end if;
190     end process;
191
192 end behavioral;
193
194
195

```